AIMERZ.ai
Aim.Act.Achieve

NOTES

Collections Framework/API - 1

Introduction Collection Framework
What is a Collection?
Key Interfaces in Collection Framework
The Key Hierarchey & Key Classes
List, Set, Concrete classes
Comparable and Comparator Interfaces

In a programmer's daily work, managing sets of data is a common task. Java provides various collections like Lists, Stacks, and Queues to handle these data sets. To make this process even easier, Java includes a Collection Framework, which uses these collections to manage data more efficiently.

In this Module, we'll explore what the Collection Framework is and its key advantages.

Introduction to Collection Framework:

The **Collection Framework** in Java is a set of tools that allows us to store and manage data easily. It simplifies the process of handling and organizing large amounts of data, making the programmer's job quicker and easier.

What is a Collection?

A collection is a group of individual data, that are grouped together to form a single unit. Collections are containers that hold multiple data as a single unit.

For example, if we store the names of all the employees in a single list and name it as Employee, it will form a collection.

The two root interfaces of java collection class are:

- ➔ Collection interface(java.util.collection)
- ➔ Map interface(java.util.Map)

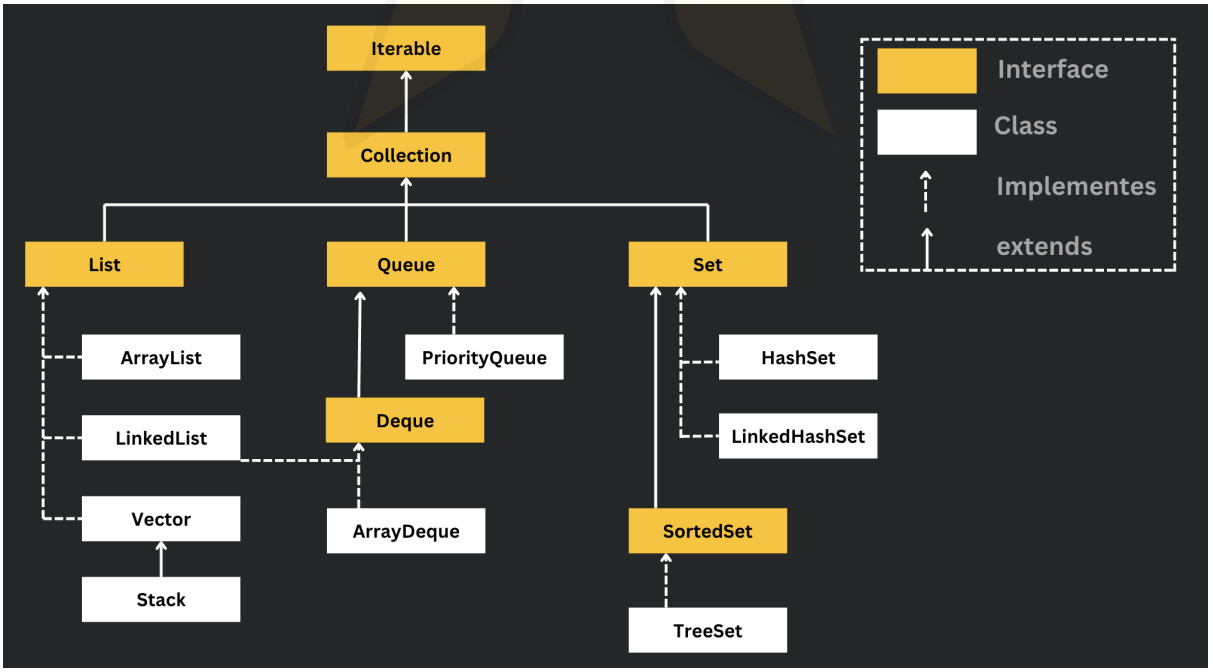
History of Collection Classes in Java

In the early versions of Java (up to JDK 1.1), developers often relied on arrays and classes like Vector, Hashtable, and Stack to manage groups of objects. While these classes were functional, they lacked a unified structure and had several limitations:

- **Inconsistency**: Each class (e.g., Vector and Hashtable) had different methods for similar operations, leading to confusion and inconsistency in code.
- **Manual Effort**: Developers had to write additional code to handle common operations like sorting, searching, or resizing collections.
- **Type Safety Issues**: These classes stored objects without type restrictions (non-generic), which meant developers had to use manual type checks and casts, often leading to runtime errors.
- **No Interoperability**: There was no common interface or architecture to allow seamless interaction between different data structures.

So to resolve these issues, Introduction of the Collection Framework (Java 2, JDK 1.2)

To address these challenges, Java 2 (JDK 1.2) introduced the Collection Framework, which provided a unified architecture for storing and manipulating groups of objects. This framework introduced interfaces like List, Set, and Map, along with their concrete implementations such as ArrayList, HashSet, and HashMap.



Let's see the **Key Interfaces in the Collection Framework**

The architecture of the Java Collection Framework is primarily built on interfaces. Interfaces are abstract data types that define collections in the framework. They allow us to work with collections without worrying about how they are stored in memory. All these interfaces are included in Java's utility package, java.util.

Interface	Description
Collection	This is the root interface and is present at the top of the Collection hierarchy and allows us to work with Sub-interfaces **List, Set, Queue**.
List	This interface extends the Collection interface and is used to store data in the form of a list. The object of List stores elements in an ordered form and allows duplicate elements.
Set	This interface extends the Collection interface and does not allow duplicate elements.
Queue	A collection that processes elements in a particular order, typically FIFO (First-In, First-Out).
SortedSet	This interface extends the Set interface and is used to handle the set of elements that are sorted.
Map	This interface does not extend any other interfaces. It is used to map the data in the form of keys and values.
SortedMap	This interface extends the Map interface and is used to maintain the keys in ascending order.
Map.Entry	This is an inner class of the Map interface that is used to represent elements(Both keys and values) on a map.

Enumeration	This is a legacy interface that is used to enumerate data one at a time in the collection of objects.

All collection frameworks in java include the following three elements:

- ➔ Interfaces
- ➔ Implementation or classes
- ➔ Collection Algorithms

And the interfaces we have already seen in the previous module
Now we will see Implementation classes in this module

Let's see the **Collection class Hierarchy and Key implementation Classes**

Parent Interface	Sub-Interface	Implementations	Description
Collection	List	ArrayList, LinkedList, Vector, Stack	Ordered collection that allows duplicates. Elements can be accessed by their index.
Collection	Set	HashSet, LinkedHashSet, TreeSet	Unordered collection (except LinkedHashSet) that does not allow duplicate elements.
Collection	Queue	PriorityQueue, LinkedList, ArrayDeque	Follows the **FIFO** (First-In-First-Out) principle. Used for handling elements in a queue-like structure.
(None - Standalone)	Map	HashMap, LinkedHashMap, TreeMap, Hashtable	Represents key-value pairs. Not part of Collection but works closely with it.

Common methods of Collection Interface;

Method	Description	Implemented By	Example
add(E e)	Adds an element to the collection.	ArrayList, HashSet, LinkedList, TreeSet, etc.	list.add("Java");
addAll(Collection<? extends E> c)	Adds all elements from the specified collection to the current collection.	ArrayList, HashSet, LinkedList, TreeSet, etc.	list.addAll(anotherList);
remove(Object o)	Removes the specified element from the collection.	ArrayList, HashSet, LinkedList, TreeSet, etc.	list.remove("Java");
removeAll(Collection<?> c)	Removes all elements in the specified collection from the current collection.	ArrayList, HashSet, LinkedList, TreeSet, etc.	list.removeAll(commonList);
retainAll(Collection<?> c)	Retains only the elements in the current collection that are contained in c.	ArrayList, HashSet, LinkedList, TreeSet, etc.	list.retainAll(sharedList);
clear()	Removes all elements from the collection.	ArrayList, HashSet, LinkedList, TreeSet, etc.	list.clear();

isEmpty()	Checks if the collection is empty.	ArrayList, HashSet, LinkedList, TreeSet, etc.	list.isEmpty();
size()	Returns the number of elements in the collection.	ArrayList, HashSet, LinkedList, TreeSet, etc.	int count = list.size();
contains(Object o)	Checks if the collection contains the specified element.	ArrayList, HashSet, LinkedList, TreeSet, etc.	list.contains("Java");
containsAll(Collection<?> c)	Checks if the collection contains all elements in the specified collection.	ArrayList, HashSet, LinkedList, TreeSet, etc.	list.containsAll(subsetList);
toArray()	Converts the collection to an array.	ArrayList, HashSet, LinkedList, TreeSet, etc.	Object[] arr = list.toArray();
iterator()	Returns an iterator to traverse the elements in the collection.	ArrayList, HashSet, LinkedList, TreeSet, etc.	Iterator<String> it = list.iterator();
stream()	Returns a sequential stream with the collection as its source.	ArrayList, HashSet, LinkedList, TreeSet, etc.	list.stream().filter(x -> x.startsWith("J")).forEach(System.out::println);

parallelStream()	Returns a parallel stream with the collection as its source.	ArrayList, HashSet, LinkedList, TreeSet, etc.	list.parallelStream().filter(x -> x.startsWith("J")).forEach(System.out::println);
equals(Object o)	Compares the specified object with the collection for equality.	ArrayList, HashSet, LinkedList, TreeSet, etc.	boolean isEqual = list.equals(anotherList);
hashCode()	Returns the hash code of the collection.	ArrayList, HashSet, LinkedList, TreeSet, etc.	int code = list.hashCode();
spliterator()	Creates a Spliterator over the elements in the collection.	ArrayList, HashSet, LinkedList, TreeSet, etc.	Spliterator<String> split = list.spliterator();

Let's get into **List, Set, Queue, and Map – Interfaces**

List Interface: A List represents an ordered collection of elements, where duplicates are allowed. It is a zero-based index structure, meaning elements can be accessed, inserted, or removed by their position.

Key Implementation Classes of List Interface are
ArrayList:
Let's see, **How ArrayList Works Internally**

Data Structure:

- ➔ Internally, an `ArrayList` uses an array (often referred to as the backing array) to store elements.

- The array is created with an initial capacity, which means a fixed amount of contiguous memory is allocated when the `ArrayList` is instantiated.

Contiguous Memory Locations:

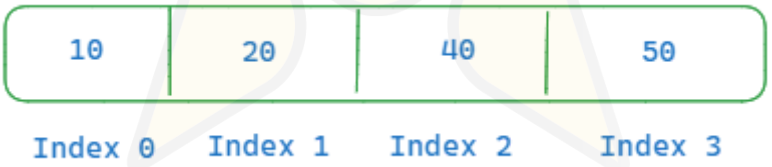
- Just like arrays, the elements in an `ArrayList` are stored in contiguous memory locations, allowing fast **random access** using an index (O(1) time complexity for `get()` operations).

Dynamic Resizing:

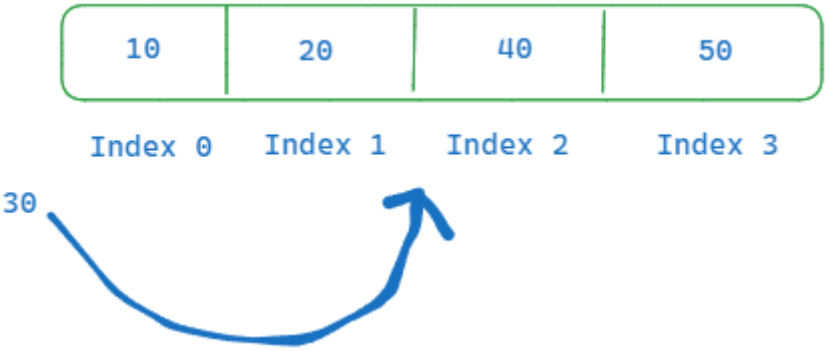
- When the `ArrayList` exceeds its current capacity, a new larger array is created, and all the elements from the old array are copied to the new array.
- By default, the new capacity is typically 1.5 times the old capacity (implementation-dependent).

Efficient Access and Inefficient Modifications:

- Accessing elements is very fast because of the contiguous memory layout.



- However, inserting or deleting elements (other than at the end) can be slow, as it may require shifting elements to maintain the order.



- Like in the above we can see that if we want to insert 30 after 20 then it will take some time do the insertion

- Follows Dynamic data structure which means the size of the collection is increased dynamically at rear end.

Real-World Analogy

Imagine a **train** with fixed compartments:

- Initially, the train has a certain number of compartments (initial capacity).
- If the train gets full, more compartments are added, and the passengers are shifted to the new train (dynamic resizing and copying).
- Accessing a specific passenger in a compartment is very quick (random access), but shifting people from one compartment to another takes time (insertion/deletion).

Features of ArrayList

- **Dynamic Resizing**: Automatically grows or shrinks as needed when elements are added or removed.
- **Random Access**: Provides fast (O(1)) access to elements using an index, thanks to its underlying array structure.
- **Allows Duplicates**: Supports duplicate elements in the list.
- **Maintains Insertion Order**: Elements are stored in the order they are added.
- **Heterogeneous Data**: Allows storing different types of objects if generics are not used (not recommended for type safety).

- **Zero Initial Size**: By default, starts with an initial capacity of **10**, but can grow dynamically.
- **Efficient for Read Operations**: Fast for accessing and reading elements due to contiguous memory allocation.
- **Slower for Modifications**: Adding or removing elements (except at the end) can be slow because of element shifting.
- **Thread-Safe Alternative**: Not synchronized by default. Use Collections.synchronizedList() or CopyOnWriteArrayList for thread safety.
- **Implements List Interface**: Provides methods like add(), remove(), get(), set(), and indexOf().
- **Null Values**: Can store multiple null values.
- **Generics Support**: Ensures type safety by specifying the type of elements during declaration.
- **Iteration Support**: Can be traversed using a for loop, foreach loop, Iterator, or ListIterator.

Constructors of ArrayList:

- **ArrayList**(): Creates an empty list with an initial capacity of 10.
- **ArrayList**(int initialCapacity): Specifies the initial capacity.
- **ArrayList(Collection<? extends E> c)**: Creates a list containing the elements of the specified collection.

When to Use ArrayList:

ArrayList: When frequent access by index is needed and insertion/deletion of elements occurs primarily at the end of the list.

Let's see the **ArrayList Code Example:**

```Java
import java.util.ArrayList;

public class ArrayListExample {
    public static void main(String[] args) {
        // Create an ArrayList of String type
        ArrayList<String> fruits = new ArrayList<>();
```

```
        // Add elements to the ArrayList
        fruits.add("Apple");
        fruits.add("Banana");
        fruits.add("Orange");

        // Access elements by index
        System.out.println("First element: " + fruits.get(0));

        // Replace an element
        fruits.set(1, "Mango");
        System.out.println("After replacement: " + fruits);

        // Remove an element by value
        fruits.remove("Orange");

        // Remove an element by index
        fruits.remove(0);

        // Loop through elements using for-each
        for (String fruit : fruits) {
           System.out.println(fruit);
        }

        // Check size
        System.out.println("Size of ArrayList: " +
fruits.size());
    }
}
```

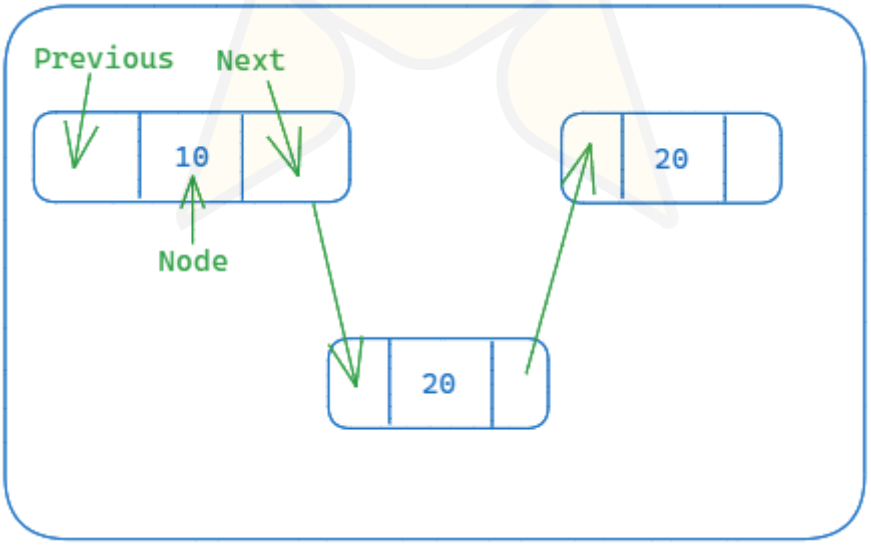
Output

```
Java
First element: Apple
After replacement: [Apple, Mango, Orange]
Mango
Size of ArrayList: 1
```

LinkedList:

Features of LinkedList

- **Doubly Linked List Implementation**: Internally implemented as a doubly linked list where each node contains data and pointers to both the previous and next nodes.



- **Dynamic Size**: No need to specify an initial size. Grows and shrinks dynamically as elements are added or removed.
- **Efficient Insertions and Deletions**: Adding or removing elements at the beginning, middle, or end is faster than ArrayList because no shifting of elements is needed.

- **Sequential Access**: Accessing elements requires traversal from the head or tail, making random access slower (O(n) time complexity).
- **Maintains Insertion Order**: Elements are stored in the order they are inserted.
- **Allows Duplicates**: Supports duplicate elements in the list.

- **Stores Null Values**: Can store multiple null values.

- **Implements Both List and Deque Interfaces**: Supports operations from both the List interface (like add(), remove(), get()) and the Deque interface (like addFirst(), addLast(), removeFirst(), removeLast()).
- **Thread-Safe Alternative**: Not synchronized by default. Use Collections.synchronizedList() for thread safety.
- **Iteration Support**: Can be traversed using Iterator, ListIterator, for loop, or foreach loop.
- **Efficient Queue and Stack Operations**: Ideal for implementing data structures like queues (FIFO) or stacks (LIFO) due to efficient add/remove operations at both ends.
- **Generics Support**: Ensures type safety by allowing you to specify the type of elements.
- **Custom Sorting**: Elements can be sorted using Collections.sort() or custom comparators.

Constructors of LinkedList:

- **LinkedList()**: Creates an empty list.
- **LinkedList(Collection<? extends E> c):** Creates a list containing the elements of the specified collection.

Real-World Use

- ➔ **ArrayList**: Use when you need fast access to elements, e.g., browsing products in an e-commerce app.
- ➔ **LinkedList**: Use when frequent insertions and deletions occur, e.g., managing a playlist where songs are added/removed frequently.

When to use LinkedList: When frequent insertions/deletions in the middle of the list are required or when you need to treat it like a queue.

LinkedList Example

```Java
import java.util.LinkedList;
public class LinkedListExample {
    public static void main(String[] args) {
        // Create a LinkedList of String type
        LinkedList<String> animals = new LinkedList<>();

        // Add elements to the LinkedList
        animals.add("Lion");
        animals.add("Tiger");
        animals.addFirst("Elephant"); // Adds element at the front
        animals.addLast("Monkey");  // Adds element at the end

        // Access elements
        System.out.println("First animal: " + animals.getFirst());
        System.out.println("Last animal: " + animals.getLast());

        // Remove elements
        animals.removeFirst(); // Removes the first element
        animals.removeLast();  // Removes the last element

        // Print all elements using a for-each loop
        for (String animal : animals) {
            System.out.println(animal);
        }
```

```
        // Check size
        System.out.println("Size of LinkedList: " + animals.size());
    }
}
```

Output

```Java
First animal: Elephant
Last animal: Monkey
Lion
Tiger
Size of LinkedList: 2
```

LinkedList Comparison with ArrayList

Feature	ArrayList	LinkedList
Data Structure	Backed by a dynamic array	Backed by a doubly linked list
Access Speed	Fast (O(1)) for random access	Slow (O(n)) for random access
Insert/Delete	Slow due to element shifting	Fast (no shifting required)
Memory	Less memory (stores only data)	More memory (stores data + pointers)
Best Use Case	Frequent retrieval	Frequent insertions/deletions

Set Interface: A Set represents a collection that does not allow duplicate elements and is typically unordered.

Common Methods in Set:

set.add("Orange"); // Adds an element

set.contains("Orange"); // Checks if "Orange" exists

set.remove("Orange"); // Removes an element

set.size(); // Returns the size of the set

Key Implementation classes of Set Interface:

HashSet, TreeSet, LinkedHashSet

Features of HashSet:

- **No Duplicate Elements**: Does not allow duplicate elements, ensuring a collection of unique values.
- **Unordered Collection**: Does not maintain the order of elements; the insertion order is not guaranteed.
- **Backed by HashMap**: Internally uses a HashMap to store elements as keys, with a dummy value associated with each key.
- **Allows Null Value**: Allows a single null element in the set.
- **Fast Operations**: Provides constant-time performance (O(1)) for basic operations like add(), remove(), and contains() (assuming a good hash function).
- **Implements Set Interface**: Implements the Set interface, ensuring no duplicates and offering methods like add(), remove(), and contains().
- **Dynamic Size**: Automatically resizes as more elements are added.
- **No Indexing**: Does not support accessing elements by an index. Traversal can only be done using iterators or enhanced for loops.

- **Not Thread-Safe**: Not synchronized by default. Use Collections.synchronizedSet() or ConcurrentHashSet for thread-safe operations.
- **Hashing Mechanism**: Uses the **hashCode()** and **equals()** methods to ensure uniqueness and locate elements efficiently.
- **Custom Objects in HashSet**: Custom objects can be stored, but you must override the hashCode() and equals() methods for proper functionality.

Constructors of HashSet::

- **HashSet()**: Creates an empty set with an initial capacity of 16 and a load factor of 0.75.
- **HashSet(int initialCapacity)**: Specifies the initial capacity.
- **HashSet(int initialCapacity, float loadFactor)**: Specifies both initial capacity and load factor.
- **HashSet(Collection<? extends E> c)**: Creates a set containing the elements of the specified collection.

Initial Capacity

- **Definition**: The initial capacity is the number of buckets a HashMap starts with when it's created. Think of it as the initial size of the map's internal storage.
- **Purpose**: Setting an appropriate initial capacity can enhance performance by reducing the need for resizing as elements are added. If you anticipate storing a large number of entries, specifying a higher initial capacity can prevent multiple resizes.

Load Factor

- **Definition**: The load factor is a measure that determines when the HashMap should increase its capacity to maintain efficient performance. It's defined as the ratio of the number of elements to the current capacity. The default load factor is 0.75, meaning the map will resize when it's 75% full.
- **Purpose**: The load factor balances time and space efficiency. A lower load factor reduces the likelihood of collisions (where

multiple keys hash to the same bucket) but increases memory usage. Conversely, a higher load factor increases the chance of collisions, which can degrade performance, but uses memory more efficiently.

Example of HashMap

Consider a HashMap with an initial capacity of 16 and a load factor of 0.75. This means:

- ➔ The map has 16 buckets initially.
- ➔ When the number of entries exceeds 12 (16 * 0.75), the map will resize—typically doubling its capacity to 32 buckets—to maintain performance.

Real-World Use
HashSet is ideal for scenarios where uniqueness is crucial:

- ➔ **Student Roll Numbers**: Ensure no duplicate roll numbers in a school database.
- ➔ **Unique Voter IDs**: Store a collection of voter IDs to ensure uniqueness in an election system.

When to Use HashSet:

HashSet: When we need high-performance, no duplicates, and don't care about order.

HashSet Example:

```Java
import java.util.HashSet;

public class HashSetExample {
    public static void main(String[] args) {
        // Create a HashSet of Integer type
        HashSet<Integer> numbers = new HashSet<>();
```

```java
        // Add elements to HashSet
        numbers.add(10);
        numbers.add(20);
        numbers.add(30);
        numbers.add(20); // Duplicate element, will be ignored

        // Iterate over elements
        for (Integer num : numbers) {
            System.out.println(num);
        }

        // Check if a number exists
        System.out.println("Contains 10: " + numbers.contains(10));

        // Remove an element
        numbers.remove(20);

        // Check size
        System.out.println("Size of HashSet: " + numbers.size());
    }
}
```

Output:

```java
20
10
30
Contains 10: true
Size of HashSet: 2
```

Comparison with Other Collections

Feature	ArrayList	HashSet	LinkedList
Duplicates Allowed	Yes	No	Yes
Order Maintained	Yes (insertion order)	No	Yes (insertion order)
Access Speed	O(1) for index-based	O(1) for contains() (on average)	O(n) for sequential access
Best Use Case	Frequent random access	Ensuring uniqueness	Frequent insertions/deletions

Features of TreeSet

- ➔ **Sorted Set Implementation**: Maintains elements in **sorted order** (ascending by default) based on their natural ordering or a custom comparator.
- ➔ **No Duplicates Allowed**: Ensures that only unique elements are stored, just like a HashSet.
- ➔ **Tree-Based Data Structure**: Internally uses a **Red-Black Tree**, a self-balancing binary search tree, for storage.
- ➔ **Implements NavigableSet**: Provides navigation methods like lower(), higher(), floor(), and ceiling() to retrieve elements relative to a given value.
- ➔ **Automatic Sorting**: Sorts elements automatically, eliminating the need for manual sorting after insertion.
- ➔ **Null Element Restriction**: Does **not allow null elements** because it uses comparisons, and null cannot be compared.
- ➔ **Dynamic Size**: Automatically resizes as elements are added or removed.
- ➔ **Slower than HashSet**: Performance is slower than HashSet because it requires additional overhead for sorting.
- ➔ **Thread Safety**: Not synchronized by default. Use Collections.synchronizedSortedSet() for thread safety.

- **Custom Sorting**: Allows custom sorting by passing a Comparator during TreeSet creation.
- **Supports Iteration**: Provides an iterator to traverse the elements in sorted order.
- **Subset and Range Operations**: Offers methods like subSet(), headSet(), and tailSet() to get specific ranges of elements.

Constructors of TreeSet:

- **TreeSet()**: Creates an empty set sorted according to the natural ordering of its elements.
- **TreeSet(Comparator<? super E> comparator)**: Specifies a comparator for order.
- **TreeSet(Collection<? extends E> c)**: Creates a set containing the elements of the specified collection, sorted according to their natural ordering.
- **TreeSet(SortedSet<E> s)**: Creates a set containing the same elements and using the same ordering as the specified sorted set.

Real-World Use

TreeSet is ideal for scenarios where sorted data is required:

- **Leaderboard System**: Ranking players or scores in a game.
- **Event Scheduler**: Storing and retrieving events in chronological order.
- **Unique Sorted Names**: Maintaining a list of names in alphabetical order.

When to use TreeSet:

TreeSet: When we need a sorted set.

TreeSet Example:

```java
import java.util.TreeSet;

public class TreeSetExample {
    public static void main(String[] args) {
        // Create a TreeSet of String type
        TreeSet<String> countries = new TreeSet<>();

        // Add elements to TreeSet
        countries.add("India");
        countries.add("USA");
        countries.add("Australia");
        countries.add("India"); // Duplicate element, ignored

        // TreeSet automatically sorts elements in natural order
        System.out.println(countries);

        // Get first and last elements
        System.out.println("First country: " + countries.first());
        System.out.println("Last country: " + countries.last());
    }
}
```

Output:

```java
Australia
India
USA
First country: Australia
Last country: USA
```

Comparison with Other Sets

Feature	HashSet	TreeSet
Order Maintained	No	Yes (sorted order)
Duplicates Allowed	No	No
Null Elements	Allows one null value	Does not allow null
Performance	Faster (O(1) for basic ops)	Slower (O(log n) for basic ops)
Use Case	Fast and unordered storage	Sorted storage with navigation

Features of LinkedHashSet

- ➔ **Preserves Insertion Order**: Maintains the order in which elements are added, unlike HashSet.
- ➔ **No Duplicates Allowed**: Ensures that only unique elements are stored, similar to HashSet.
- ➔ **Uses Hash Table and Linked List**: Internally combines a **hash table** (for fast access) and a **linked list** (to maintain order).
- ➔ **Faster Lookups**: Provides constant-time performance for basic operations like add(), remove(), and contains().
- ➔ **Null Element Allowed**: Allows at most one null value.
- ➔ **Dynamic Size**: Automatically grows as elements are added.
- ➔ **Not Thread-Safe**: Not synchronized by default. Use Collections.synchronizedSet() for thread safety in multithreaded environments.
- ➔ **Ideal for Ordered Unique Data**: Best suited when you need a collection that retains insertion order and avoids duplicates.
- ➔ **Iteration Order Matches Insertion Order**: Iterating through a LinkedHashSet returns elements in the same order they were added.

- **Performance**: Slightly slower than HashSet due to the overhead of maintaining the linked list.

Constructors of LinkedHashSet::

- **LinkedHashSet()**: Creates an empty set with the default capacity (16) and load factor (0.75).
- **LinkedHashSet(int initialCapacity)**: Specifies the initial capacity.
- **LinkedHashSet(int initialCapacity, float loadFactor)**: Specifies both initial capacity and load factor.
- **LinkedHashSet(Collection<? extends E> c)**: Creates a set containing the elements of the specified collection.

Real-World Example

LinkedHashSet is ideal when order matters:

- **Browser History:** Storing unique URLs in the order visited.
- **Product Wishlist:** Maintaining products in the order added by the user.
- **Event Logs:** Storing unique event names in the order they occurred.

When to use LinkedHashSet:

LinkedHashSet: When you need to maintain the insertion order while avoiding duplicates.

LinkedHashSet Example

```Java
import java.util.LinkedHashSet;

public class LinkedHashSetExample {
```

```java
    public static void main(String[] args) {
        LinkedHashSet<String> cities = new LinkedHashSet<>();

        // Adding elements
        cities.add("New York");
        cities.add("Los Angeles");
        cities.add("Chicago");
        cities.add("New York"); // Duplicate, will not be added

        // Displaying elements in insertion order
        System.out.println(cities); // Output: [New York, Los
Angeles, Chicago]

        // Checking if an element exists
        System.out.println("Contains Chicago? " +
cities.contains("Chicago")); // Output: true

        // Removing an element
        cities.remove("Los Angeles");
        System.out.println(cities); // Output: [New York, Chicago]
    }
}
```

Output:

```java
[New York, Los Angeles, Chicago]
Contains Chicago? true
[New York, Chicago]
```

Comparison with Other Sets

Feature	HashSet	TreeSet	LinkedHashSet
Order Maintained	No	Yes (sorted order)	Yes (insertion order)
Duplicates Allowed	No	No	No
Null Elements	Allows one null value	Does not allow null	Allows one null value
Underlying Data Structure	Hash Table	Red-Black Tree	Hash Table + Linked List
Performance	Fastest	Slower (due to sorting)	Slightly slower than HashSet

Comparable and Comparator Interfaces in Java

Both the Comparable and Comparator interfaces in Java are used for sorting objects. However, they are used in different scenarios and have distinct purposes.

- ➔ **Comparable**: It is used to define the natural ordering of objects. Classes implementing Comparable can be compared using their natural order (like ascending or lexicographical order).
- ➔ **Comparator**: It is used when you need more flexibility in comparing objects, such as when you want to sort objects in a different order than the natural one or when the class does not implement Comparable.

Let's begin with **Comparable** Interface

Comparable Interface: The Comparable interface is present in **java.lang** package and provides a way for objects of a class to be compared with each other based on one attribute, known as the natural ordering of the objects.

Understanding the Comparable Interface

- **Purpose**: Comparable defines the *natural ordering* for objects of a class.
- **Use Case of Comparable:**: Use Comparable when the primary way to compare objects is already decided, such as sorting students by roll numbers, employees by ID, etc.
- **Location**: Found in the java.lang package.

How Comparable Works

A class implementing Comparable must override the **compareTo()** method:

Basic Syntax:

```Java
public interface Comparable<T> {

      public int compareTo(T o);

}
```

Syntax Explanation

- **T**: Represents the type of object being compared.
- The method **compareTo(T o)** compares the current object (this) with the object passed as an argument (o).

How compareTo() Works:

- **Returns 0**: If the current object is **equal** to the passed object.
- **Returns a negative integer**: If the current object is **less than** the passed object.

- **Returns a positive integer**: If the current object is **greater than** the passed object.

Real-World Analogy for Comparable

Imagine a **student leaderboard** in a school:

- The leaderboard is always sorted by roll numbers (default order).
- Comparable ensures every student knows how to compare themselves with another student based on this predefined criterion.

Let's see an Example: We have a class Student and want to compare students based on their **roll number**.

Example: Using Comparable

Sort students by their roll number.

```Java
import java.util.ArrayList;
import java.util.Collections;

class Student implements Comparable<Student> {
    int rollNo;
    String name;

    public Student(int rollNo, String name) {
        this.rollNo = rollNo;
        this.name = name;
    }

    // Implementing the compareTo method to compare by roll
number
    @Override
    public int compareTo(Student otherStudent) {
```

```
        return this.rollNo - otherStudent.rollNo;  //
Ascending order by roll number
    }

    @Override
    public String toString() {
        return "Student{rollNo=" + rollNo + ", name='" + name
+ "'}";
    }
}

public class ComparableExample {
    public static void main(String[] args) {
        ArrayList<Student> students = new ArrayList<>();
        students.add(new Student(102, "Alice"));
        students.add(new Student(101, "Bob"));
        students.add(new Student(103, "Charlie"));

        // Sorting students by roll number using Comparable
        Collections.sort(students);

        // Display sorted students
        for (Student student : students) {
           System.out.println(student);
        }
    }
}
```

Output

```
Java
Student{rollNo=101, name='Prabir'}
Student{rollNo=102, name='Sukumar'}
Student{rollNo=103, name='Deepak'}
```

Real-Time Usage of Comparable

- ➔ **Sorting Files by Size**: File objects can implement Comparable to sort files by size.
- ➔ **Leaderboard in Games**: Players can be sorted by score using Comparable.

Now let's see the Comparator Interface

Comparator Interface: The Comparator interface is present in **java.util** package and provides a way to compare two objects of the same class based on different attributes. It is used when you want more control over the sorting mechanism.

Understanding the Comparator Interface

- ➔ **Purpose**: Comparator provides more flexibility and allows sorting objects using multiple attributes or custom logic.
- ➔ **Location**: Found in the java.util package.

How Comparator Works

Comparator requires overriding the **compare()** method:

Syntax:

```Java
public interface Comparator<T> {
    public int compare(T o1, T o2);
}
```

Syntax Explanation

- ➔ **T**: Represents the type of object being compared.

- The method **compare(T o1, T o2)** compares two objects (o1 and o2).

How compare() method Works:

- **Returns 0**: If both objects are **equal**.
- **Returns a negative integer**: If the first object is **less than** the second.
- **Returns a positive integer**: If the first object is **greater than** the second.

Use Case of Comparator:

- When you want to sort objects based on **multiple attributes** or need **custom sorting** (e.g., sorting by **name**, **salary**, or any other attribute).
- Unlike Comparable, you do not need to modify the class to implement the Comparator interface. You can have multiple Comparator implementations for the same class.

Real-World Analogy for Comparator

Imagine sorting a **bookstore inventory**:

- Customers can ask for sorting by price, author, or genre.
- Comparator provides different sorting strategies, just like different customer preferences.

Example: Using Comparator :
Scenario: Sort employees by salary and then by name.

```Java
import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;
```

```java
class Employee {
    int id;
    String name;
    double salary;

    public Employee(int id, String name, double salary) {
        this.id = id;
        this.name = name;
        this.salary = salary;
    }

    @Override
    public String toString() {
        return "Employee{id=" + id + ", name='" + name + "',
salary=" + salary + "}";
    }
}

class SalaryComparator implements Comparator<Employee> {
    @Override
    public int compare(Employee e1, Employee e2) {
        return Double.compare(e1.salary, e2.salary);  //
Ascending order by salary
    }
}

class NameComparator implements Comparator<Employee> {
    @Override
    public int compare(Employee e1, Employee e2) {
        return e1.name.compareTo(e2.name);  // Alphabetical
order by name
    }
}

public class ComparatorExample {
    public static void main(String[] args) {
        ArrayList<Employee> employees = new ArrayList<>();
        employees.add(new Employee(1003, "John", 50000));
```

```java
        employees.add(new Employee(1001, "Alice", 70000));
        employees.add(new Employee(1002, "Bob", 60000));

        System.out.println("Sorting by salary:");
        Collections.sort(employees, new SalaryComparator());
        for (Employee e : employees) {
           System.out.println(e);
        }

        System.out.println("\nSorting by name:");
        Collections.sort(employees, new NameComparator());
        for (Employee e : employees) {
           System.out.println(e);
        }
    }
}
```

Output

```java
Sorting by salary:
Employee{id=1003, name='John', salary=50000.0}
Employee{id=1002, name='Bob', salary=60000.0}
Employee{id=1001, name='Alice', salary=70000.0}

Sorting by name:
Employee{id=1001, name='Alice', salary=70000.0}
Employee{id=1002, name='Bob', salary=60000.0}
Employee{id=1003, name='John', salary=50000.0}
```

Real-Time Usage of Comparator

- ➔ **E-Commerce Filters**: Sort products by price, ratings, or popularity.
- ➔ **Event Management**: Sort participants by registration time or performance score.
- ➔ **Custom Sorting Logic in Databases**: Sort records fetched from a database dynamically based on user preferences.

Key Differences Between Comparable and Comparator

Feature	Comparable	Comparator
Package	java.lang	java.util
Method to Implement	compareTo(T o)	compare(T o1, T o2)
Sorting Logic	Defines **natural ordering** of a class.	Can define **multiple ways** to compare objects.
Modifies Class	Requires modifying the class to implement Comparable.	No need to modify the class. Separate comparator classes can be created.
Use Case	Single default sorting based on one attribute.	Multiple sorting criteria or sorting without changing the class.
Example	Sorting Students by **roll number**.	Sorting Employees by **salary** or **name**.

THANK YOU

AIMERZ.ai

Aim.Act.Achieve

Stay updated with us!

Aimerz

Aimerz

Aimerz DS

Aimerz FSD

Vishwa Mohan

Vishwa Mohan

vishwa.mohan.singh

Vishwa Mohan